

TAAM Artifact Management System: CSCB07 S26



Project Overview

The goal of this project is to build an Android application for managing and viewing artifacts from the TAAM collection. The application must be built as a Java Android application using XML layouts, Firebase Realtime Database, and cloud image storage.



User Roles

The application must support two types of logged-in accounts: regular users and admin users.

A regular user can:

- Log in using their email and password
- View the artifact home page
- Search and browse artifacts
- Open an expanded artifact view for full artifact details
- View artifact likes and comments
- Like and unlike artifacts
- Add comments to artifacts
- Save artifacts to their own saved collection
- View saved artifacts
- Log out

An admin user can do everything a regular user can do, and additionally:

- Add new artifacts
- Edit existing artifacts
- Delete artifacts
- Delete comments on artifacts



Artifact Data Model

Each artifact must include the following fields:

- Lot number (mandatory, unique identifier)
- Artifact name (mandatory)

- Description (mandatory)
- Category (mandatory)
- Material (mandatory)
- Dynasty/Period (mandatory)
- Cultural origin
- Dimensions
- Condition report
- Current location
- Acquisition method
- Provenance
- Accession number
- Notes
- Image

Category, Material, and Dynasty/Period will have fixed options to choose from:

Category Options:

- Painting and Calligraphy
- Ceramics
- Bronze Ware
- Lacquerware
- Jade Ware
- Enamel Ware
- Glassware
- Furniture
- Embroidery and Textiles
- Documents and Archives
- Gold and Silverware
- Clocks and Watches
- Religious Artifacts
- Daily-use Items
- Weaponry
- Miscellaneous Artifacts

Material Options:

- Bronze
- Stone
- Wood
- Jade
- Ceramic
- Lacquerware
- Ivory
- Gold
- Silver
- Iron
- Mixed Media

Dynasty/Period Options:

- Shang Dynasty (c. 1600-1046 BCE)
- Western Zhou Dynasty (c. 1046-771 BCE)
- Eastern Zhou Dynasty (770-256 BCE)
- Qin Dynasty (221-206 BCE)
- Han Dynasty (206 BCE-220 CE)
- Three Kingdoms Period (220-280 CE)
- Jin Dynasty (266-420 CE)
- Southern and Northern Dynasties (420-589 CE)
- Sui Dynasty (581-618 CE)
- Tang Dynasty (618-907 CE)
- Five Dynasties and Ten Kingdoms (907-960 CE)
- Song Dynasty (960-1279 CE)
- Liao Dynasty (907-1125 CE)
- Jin Dynasty (1115-1234 CE)
- Yuan Dynasty (1271-1368 CE)
- Ming Dynasty (1368-1644 CE)
- Qing Dynasty (1644-1912 CE)
- Republic of China Period (1912-1949 CE)

Login/Sign-up Requirements

The app will launch to the login/sign-up screen. It should utilize:

- MVP (model, view, presenter) code structure
- Unit tests for login/signup presenter logic where applicable

Note that MVP and unit tests are only needed for the login/signup portion, not the entire app.

New users can create a new account with an email, username, and password. Existing users can log in with their email and password.

Suggested Resources: Firebase Authentication, EditText, JUnit.



Home Page Requirements

The home page must support:

- Viewing/browsing all artifacts
- Keyword search (i.e. the search text is a substring match for any field of an artifact)
- Navigation to the expanded artifact view
- Viewing saved artifacts for regular users
- Admin access to Add Artifact from the admin home page
- [Bonus] The home page may support pagination for 12 artifacts, 24 artifacts, or all artifacts per page. If pagination is implemented, the selected pagination option should be saved locally using SharedPreferences so the app remembers the user's previous choice.

Suggested Resources: RecyclerView, RecyclerView.Adapter, LinearLayoutManager/GridLayoutManager, EditText + TextWatcher or SearchView.



Expanded Artifact View Requirements

The expanded view must display the full artifact record, including its image and all fields from the data model.

The expanded view must also include:

- Number of likes
- Like/unlike control
- Artifact comments
 - Each comment should include the text and the commenter's username
 - Including comment timestamp and/or sorting by it is optional, not required
- Add comment control

- Save/unsave artifact control
- Artifact edit control for admin
- Artifact delete control for admin
 - Before deleting, a confirmation warning should display
 - Nice-to-have: When deleting an artifact, the app may also delete its uploaded image from cloud storage. This is not required as long as the deleted artifact is no longer displayed in the app.
- Comment delete control for admins
- [Bonus] The expanded artifact view may include a related artifacts section. These will display a few artifacts that are related to the current artifact in view. Related artifacts can be chosen using a simple similarity approach, such as comparing category, material, dynasty/period, cultural origin, etc.

Suggested Resources: ImageView, Glide, AlertDialog



Add and Edit Artifact Requirements

Admin users should be able to add new artifacts and edit existing artifacts.

- The Add/Edit Artifact screen must include all artifact fields listed in the data model.
- When adding an artifact, the app must validate that mandatory fields are not empty, and the lot number is unique.
- When editing an artifact, admin users may update all fields except the lot number.
- The Add/Edit Artifact screen must support selecting an image from the Android device and uploading it to cloud storage. For cloud image storage, you may use any suitable cloud storage provider. A no-credit-card storage option and setup code will be provided in the appendix.
- Nice-to-have: Like deletion, when replacing an artifact image, the app may delete the previous image from cloud storage. This is not required as long as the artifact record is updated to display the new image.

Suggested Resources: EditText, Spinner/AutoCompleteTextView, Android Photo Picker with ActivityResultLauncher, Firebase Realtime Database, cloud storage API.

Appendix: Supabase Image Storage Option

Supabase Storage is one suggested no-credit-card option for cloud image storage. Since this project uses Java Android, the provided helper uses OkHttp to send an HTTPS upload request to Supabase Storage and returns the uploaded image's public URL. Uploaded images are saved using timestamp-based file paths so that new uploads receive unique URLs.

Steps

- Configure your `strings.xml` file with your `supabase_url`, `supabase_anon_key`, and `supabase_image_bucket` values.
- Also add `OkHttp` to your Gradle dependencies.
- Copy the `SupabaseImageUploader.java` file into your project.

Example Usage

```
private SupabaseImageUploader imageUploader;

imageUploader = new SupabaseImageUploader(requireContext());

imageUploader.uploadImage(uri, lotNumber, new
SupabaseImageUploader.UploadCallback() {

    @Override

    public void onSuccess(String publicUrl) {

        // Handle successful upload

    }

    @Override

    public void onError(String message) {

        // Handle upload error

    }

});
```